## Topic : spendwise

**Team number:13**

**Team head: SOBIKA E**

**(3C617FCDEF4F4928F0140C2E99EBEC7D)**

**Team members:**

**SALINI A**

**(5D1B6176072EA5CAE5F47C26EB55BDCD)**

**SHALINI K**

**(57768CA5A59825183C74F9664EB7F42A)**

**SHALINI M**

**(0B7690CA8EABF9687674AB20E1799889)**

**HALINI M S**

**(F60DCB2A87A985CB36C464C7C824590F)**

# Abstract

Personal finance management has become increasingly important in the digital era, where individuals handle multiple financial transactions daily. Mobile and web-based financial applications help users monitor income, expenses, and savings. This project presents **SpendWise**, a backend-focused personal expense tracking application developed using **Node.js, MongoDB, and JSON Web Tokens (JWT)**.

The goal of SpendWise is to provide a simple and secure backend system that enables users to record expenses, categorize transactions, and analyze financial habits. The application demonstrates modern backend development practices such as **RESTful API architecture, authentication, database management, and security mechanisms**.

Node.js is used to build the server-side logic, while MongoDB serves as the NoSQL database for storing financial records. JWT authentication ensures secure user login and session management. The project focuses on backend learning concepts including **API development, data modeling, request handling, and authentication mechanisms**.

SpendWise serves as a learning project for students and developers interested in understanding how financial applications manage transaction data securely and efficiently.

# 1. Introduction

Financial management is a critical aspect of personal life. With the increasing use of digital payments and online banking, individuals often find it difficult to keep track of their spending

habits. Personal finance applications help users monitor their expenses and maintain better financial discipline.

Expense tracking applications allow users to record their income, categorize expenses, and analyze spending patterns. Many popular applications such as Mint, YNAB, and PocketGuard provide these services. However, understanding how such systems work at the backend level is essential for software developers.

SpendWise is designed as a **simple backend learning project** to demonstrate how financial data can be handled using modern web technologies. The project focuses on backend development concepts rather than complex front-end interfaces.

The core technologies used include:

- **Node.js** for server-side development
- **Express.js** for API routing
- **MongoDB** for storing financial data
- **JSON Web Tokens (JWT)** for authentication

The project demonstrates how users can register, login, add expenses, view transaction history, and manage financial data securely.

## 2. Objectives of the Project

The primary objectives of the SpendWise project are:

1. To design a simple backend system for managing personal financial transactions.
2. To implement secure user authentication using JSON Web Tokens.
3. To develop RESTful APIs for handling expense data.
4. To demonstrate database operations using MongoDB.
5. To provide a learning platform for understanding backend development concepts.

Additional objectives include:

- Improving knowledge of Node.js server architecture
- Understanding data modeling using NoSQL databases
- Implementing secure authentication and authorization
- Managing user-based financial data efficiently

# 3. Literature Review

Several studies and research works have examined personal finance management systems and financial technology applications.

Talathi (2025) introduced a prototype expense tracker system that focuses on managing personal financial transactions efficiently. The study highlights the importance of secure data handling and simplified expense monitoring.

Stefanov et al. (2024) discussed the development of personal finance management applications that allow users to track spending habits and analyze financial patterns. Their research emphasized the role of digital applications in improving financial awareness.

Bitrián, Buil, and Catalán (2021) explored gamification techniques in personal finance applications. Their findings suggest that integrating interactive elements such as rewards and financial goals can increase user engagement and encourage better financial habits.

Carlin, Olafsson, and Pagel (2021) analyzed the adoption of financial technology applications and found that ease of use, trust, and security are key factors influencing user adoption.

Alenazi (2023) examined transaction management in financial applications, highlighting the importance of secure authentication and efficient database management systems.

These studies provide the foundation for the development of SpendWise as a simplified backend model for financial applications.

# 4. System Architecture

The SpendWise system follows a **client-server architecture**.

**Components of the system:**

1. **Client Application**
2. **Backend Server**
3. **Database**

The client sends requests to the backend server through REST APIs. The backend processes the request and interacts with the MongoDB database to retrieve or store information.

# 5. Technologies Used

**Node.js**

Node.js is a JavaScript runtime environment used for building scalable server-side applications. It enables developers to use JavaScript for backend development.

**Express.js**

Express.js is a lightweight web framework for Node.js that simplifies API development and request handling.

**MongoDB**

MongoDB is a NoSQL database that stores data in JSON-like documents. It provides flexibility and scalability for modern web applications.

**JSON Web Token (JWT)**

JWT is used for secure authentication. It allows the server to verify user identity without storing session data.

**Mongoose**

Mongoose is an Object Data Modeling (ODM) library used to manage MongoDB data structures in Node.js.

## 6. Database Design

The database stores two primary types of data:

1. **User Information**
2. **Expense Transactions**

**User Schema**

```
User
{
  name: String,
  email: String,
  password: String,
  createdAt: Date
}
```

**Expense Schema**

```
Expense
{
  userId: ObjectId,
  title: String,
  amount: Number,
  category: String,
  date: Date
}
```

## 7. API Design

SpendWise provides several REST APIs.

**User Authentication APIs**

| Method | Endpoint | Description |
|---|---|---|
| POST | /register | Register new user |
| POST | /login | Login user |
| GET | /profile | Get user profile |

**Expense Management APIs**

| Method | Endpoint | Description |
|---|---|---|
| POST | /expenses | Add new expense |
| GET | /expenses | Get all expenses |
| PUT | /expenses/:id | Update expense |
| DELETE | /expenses/:id | Delete expense |

## 8. Implementation

**The backend system is implemented using Node.js and Express.**

**Server Setup Example**

```javascript
const express = require('express');
const app = express();

app.use(express.json());

app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

**JWT Authentication Example**

```javascript
const jwt = require('jsonwebtoken');

const token = jwt.sign(
  { userId: user._id },
  process.env.JWT_SECRET,
  { expiresIn: "1h" }
);
```

**JWT tokens are generated when the user logs in and must be included in future API requests.**

## 9. Security Mechanisms

Security is an important part of financial applications.

SpendWise implements the following security measures:

- Password hashing using bcrypt
- Token-based authentication using JWT
- Protected routes using middleware
- Input validation for API requests

These techniques help protect user data and prevent unauthorized access.

## 10. Testing

The APIs were tested using tools such as:

- Postman
- REST client tools

Testing ensures that all endpoints work correctly and handle errors properly.

Example test cases include:

- User registration validation
- Login authentication
- Expense creation
- Expense retrieval

## 11. Advantages of SpendWise

- Simple backend learning project
- Demonstrates modern web development practices
- Secure authentication using JWT
- Scalable database using MongoDB
- Useful for learning REST API development

## 12. Limitations

The project also has several limitations:

- No advanced analytics features
- No budgeting tools
- No mobile application interface
- Limited data visualization

Future improvements can address these limitations.

## 13. Future Enhancements

Several features can be added in the future:

- Expense analytics dashboard
- Budget planning system
- Mobile application integration
- AI-based financial recommendations
- Gamification features for saving goals

## 14. Conclusion

SpendWise demonstrates the development of a simple backend system for managing personal financial data. The project shows how modern web technologies such as Node.js, MongoDB, and JWT can be used to create secure and scalable financial applications.

Through this project, developers can understand important backend concepts such as authentication, API design, and database management. Although SpendWise is a basic

implementation, it provides a strong foundation for building more advanced personal finance applications.